PROJECT REPORT • TEAM NO. 3

RecallNet

Supply Chain Traceability
and Product Recall System

PROJECT AUTHORS

I.V. THANVITHA	KULAKARNI VIJAYEENDRA
SRN: PES2UG24CS197	SRN: PES2UG24CS244

Submission	Date
Project Report	12 November 2026

A database-driven application for supply-chain traceability,
validated product recalls, and operational audit history.

Abstract

RecallNet is a web application for tracking production batches through a supply chain and managing product recalls. It connects a relational database for operational records with a document database for an append-only history of actions. Users can review batches, inspect supplier and distribution traceability, open recalls, create shipment drafts, and dispatch validated movements according to their assigned role. The project combines database rules with server-side API checks so that critical actions are validated before records are changed. The application was built with Next.js, React, TypeScript, MySQL, and MongoDB.

1. Introduction

A recall requires an organisation to identify affected production, determine where the product has moved, and record the steps taken. When batch, shipment, and recall details are held in disconnected records, tracing the distribution path and reviewing past actions becomes difficult. RecallNet provides one interface for these workflows and preserves the distinction between current operational data and historical audit evidence.

MySQL is the authoritative store for products, production batches, shipment movements, recalls, quantities, users, and sessions. MongoDB stores audit documents that describe completed actions and refer back to MySQL identifiers. This arrangement avoids maintaining two editable copies of business records.

2. Objectives

- Maintain structured records for products, production batches, supply-chain partners, shipments, and recalls.
- Trace a production batch from raw-material suppliers through distribution movements.
- Validate shipment dispatches against batch status, available quantity, recall state, and route relationships.
- Record recall impact and preserve an audit history of operational events.
- Provide authenticated, role-aware pages and API routes for the required workflows.

3. System Design and Technology

The application uses the Next.js App Router. Server-side pages and API route handlers access the databases through shared connection modules. React components provide the interactive workspace, while TypeScript is used across the web application. The project dependencies include the MySQL and MongoDB Node.js drivers, Argon2 for password hashing, and Next.js 16 with React 19.

3.1 Data storage

MySQL stores structured records and relationships. The schema scripts define the relational model, demonstration DDL, seed data, query examples, advanced database logic, and the authentication and role-based access migration. MongoDB stores documents in the `audit_events` collection. Each event includes an event type, UTC timestamp, actor and role, entity type and MySQL identifier, event details, and a request identifier where available.

3.2 Main web areas

The interface includes an operations overview, batch search and detail pages, recall history and impact pages, and a shipment workspace. Batch detail views combine production information, raw-material composition and supplier information, and downstream distribution. The shipment workspace supports draft creation and dispatch, and displays related audit history for users with the relevant permission.

4. Database Logic and Business Rules

The MySQL implementation uses joins, views, stored procedures, triggers, and transactional checks to preserve relationships and validate business operations. The required query script demonstrates selection and ordering, a supplier-to-retailer join across shipment legs, grouped batch totals, a subquery comparing batch quantity with a product average, and movement totals relevant to recall impact.

The `v_active_recall_impact` view combines open or in-progress recalls with affected batches and product names. The `v_batch_distribution` view links batches to shipment items, shipment status, source locations, destinations, and quantities. The `sp_trace_batch` procedure returns production information, raw-material composition, and distribution movements for a batch.

The `sp_dispatch_shipment` procedure validates draft status and shipment contents, locks relevant records, checks completed batch status and non-cancelled recalls, confirms the available quantity at the source, and validates the parent shipment for an onward distributor movement. It updates the shipment only after the checks succeed. Shipment triggers restrict invalid state changes, prevent changes to dispatched history, and generate dispatch and delivery timestamps using database time. The `sp_open_recall` procedure validates the recall request, prevents duplicate active recalls for a batch, records the affected quantity, and marks the production batch as recalled within a transaction.

MongoDB audit events are appended after the related MySQL operation commits. They are historical descriptions, not the current state of a batch, recall, or shipment. The project documentation notes that production deployments should add durable monitoring or retry handling for audit write failures.

5. Application Features

- Authentication with server-side sessions, logout, CSRF protections, rate limiting, and Argon2id password hashes.
- Role and permission checks for administrators, quality managers, warehouse managers, and distributors.
- Dashboard summaries and batch search with status filtering and pagination.
- Batch detail and traceability views covering suppliers, raw materials, and downstream shipment movements.
- Recall creation for authorized users and recall impact views showing affected quantities and distribution.
- Shipment draft creation and validated dispatch through the database procedure.
- MongoDB audit history for shipment and recall actions, subject to the user’s audit permissions.
- Loading, error, retry, and form feedback states in the web interface.

There is no public registration endpoint and no default administrator password. An administrator is created through the project’s interactive setup script. The current project documentation also identifies limits: warehouse accounts are not assigned to an individual warehouse, and user provisioning beyond the initial administrator and warehouse account is outside the current interface.

6. Work Completed by the Authors

I.V. Thanvitha | Database and backend work

The first contribution established the project structure and implemented the MySQL schema and operational workflow, database query and procedure support, MongoDB audit integration, API routes, authentication, sessions, and role-based access control. The backend checks permissions on the server and uses database procedures for recall opening and shipment dispatch. This work is represented by the database, MySQL API, MongoDB audit, and authentication branches and their related commits.

Kulakarni Vijayeendra | Frontend work

The second contribution completed the application-facing workflows: the shared workspace and dashboard, batch search and traceability pages, recall management and impact views, shipment workspace and audit viewer, and route documentation. The frontend includes role-aware navigation and page access, connects to the application APIs, and provides loading, error, retry, and submission feedback states. The work is represented by the feature/frontend-work branch and its pull request to main.

7. Verification

The available verification results for the current implementation are as follows:

- ESLint completed successfully.
- The TypeScript check completed successfully.
- The Next.js production build completed successfully.
- A local smoke check returned HTTP 200 for `/login`.
- A local request to `/api/health` returned HTTP 200 and reported both MySQL and MongoDB as connected.

The repository does not define an automated test script, and no end-to-end test suite is included. The health endpoint confirms database connectivity; it does not by itself prove every database workflow or user role. SQL files describe the schema and demonstrations, and the project setup instructions specify the order in which a fresh development database should be initialized. The report therefore does not claim that every workflow has been exercised end to end.

8. Conclusion

RecallNet brings together relational supply-chain records, database-enforced movement and recall rules, role-aware application access, and a MongoDB audit trail. Its main demonstration path is to locate a batch, inspect its source and distribution route, review or open a recall, and create or dispatch a shipment when the database validations allow it. The implementation has passed code quality, type, build, and local database connectivity checks. Further work can add automated integration tests, more complete user administration, and durable handling for audit write failures in a production deployment.

References

RecallNet project source and documentation: `database/mysql/README.md`, `database/mongodb/README.md`, `web/README.md`, and `web/AUTHENTICATION.md`. Implementation details are based on the accompanying SQL scripts and web application source.